

Tiny Second-hand Shopping Platform

시큐어 코딩 과제 · 개발 전 과정 보고서

핵심 목표	기능 완성도와 보안 통제를 함께 검증할 수 있는 교육용 중고거래 플랫폼
기술	Python 표준 HTTP 서버 · SQLite · Vanilla JavaScript · Docker · Caddy
검증	13개 자동 테스트 · Bandit · 데스크톱/모바일 화면 QA · Docker Compose 구성 검사
저장소	https://github.com/i-Zer0/white-market-secure-coding

01

요구사항 분석

강의 자료는 회원가입, 상품 등록·조회, 사용자 간 소통, 악성 사용자·상품 차단, 송금, 검색, 관리자 통제를 필수 범위로 제시한다. White Market은 이 요구를 유저·상품·거래·운영의 네 영역으로 재구성하고, 송금은 실제 금융정보를 저장하지 않는 거래 약속·상태 이력으로 범위를 조정했다.

기능 요구사항

영역	필수 요구	구현 결과
유저	가입·로그인·프로필·조회·차단	전화번호 본인인증, GPS 동네, 프로필 사진, 별점, 차단, 탈퇴·개인정보 다운로드
상품	등록·조회·검색·관리	다중 사진·대표 지정·순서 변경, 가격/카테고리/거리/상태 검색, 찜·최근 본 상품·알림
소통	전체/1:1 대화, 신고	상품별 1:1 실시간 채팅, 읽음·접속 상태·이미지, 가격 제안·거래 약속, 신고
거래	상태와 이력	요청→판매자 수락→예약→구매자 완료, 안전 체크리스트, 후기·별점, 증빙 이력
관리자	전체 요소 관리	회원 정지, 상품 삭제, 신고 처리, 공지 CRUD, 통계, 감사·오류·백업 관리

비기능·보안 요구사항

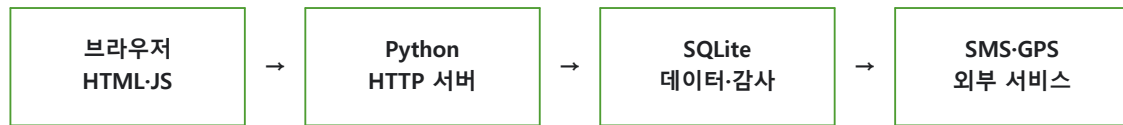
- 입력 검증, 최소 권한, 객체 소유권 검증, CSRF-XSS-SQL Injection 방어를 서버에서 강제한다.
- 비밀번호와 백업은 각각 PBKDF2-HMAC-SHA256 및 AES-GCM으로 보호하며 키는 환경 변수로 분리한다.
- 로그인·인증·채팅·검색·신고·이미지 업로드의 과도한 요청을 IP와 계정 기준으로 제한한다.
- 모바일 390px부터 데스크톱까지 가로 넘침과 이미지·설명 겹침이 없고, 키보드 포커스와 대체 텍스트를 제공한다.
- DB·백업·로그·개인 업로드·환경 변수는 Git에 포함하지 않고 Actions에서 테스트와 보안 검사를 반복한다.

범위 판단: 실제 송금과 실 SMS 계약은 금융·외부 공급자 책임이 커서 교육용 앱에 직접 내장하지 않았다. 대신 거래 당사자·상태·약속·채팅 증빙을 보존하고, SMS는 HTTPS Webhook 인터페이스와 운영 모드 강제 검증까지 구현했다.

02

시스템 구조

구성 요소와 신뢰 경계



계층	책임	핵심 파일
표현	서버 렌더링 HTML, 반응형 CSS, 실시간 채팅·이미지 처리	app.py, static/style.css, static/*js
응용	인증·상품·거래·신고·관리 흐름과 권한 검사	app.py의 App 라우터와 핸들러
데이터	트랜잭션 기반 SQLite, FK, 인덱스, 감사·이력 보존	app.py:789-1084
운영	HTTPS 리버스 프록시, 환경 변수, 영속 볼륨, 자동 검사	Dockerfile, compose.yaml, deploy/, .github/workflows/

요청 처리 원칙

- 라우터가 메서드·경로를 허용 목록으로 매핑하고 알 수 없는 경로는 일관된 404 화면으로 처리한다.
- POST는 중앙 CSRF 검증을 통과해야 하며, 기능별 핸들러가 로그인·관리자·당사자 권한을 다시 검사한다.
- DB 연결은 요청 단위 context manager로 열고 성공 시 commit, 예외 시 rollback 한다.
- 예외는 사용자에게 내부 정보를 노출하지 않고 요청 ID와 서버 오류 로그에 분리 기록한다.
- 모든 HTML 응답에 CSP, 클릭재킹·리퍼러·MIME 보호 헤더를 적용한다.

배포 데이터 분리

컨테이너 이미지는 소스와 기본 정적 자산만 포함한다. SQLite·암호화 백업은 /app/data, 프로필·상품·채팅 업로드는 각각 별도 Docker 볼륨에 저장한다. 코드 재배포와 사용자 데이터 수명을 분리해 데이터 손실과 이미지 덮어쓰기를 방지한다.

03

데이터베이스 설계

핵심 엔터티

그룹	테이블	관계·목적
계정	users, sessions, login_events, user_recovery_codes	사용자 1:N 세션·로그인 기록·복구 코드
상품	products, product_images, favorites, recent_views, keyword_alerts	판매자 1:N 상품, 상품 1:N 사진, 사용자 N:M 찜
소통	messages, chat_actions, chat_action_history	발신·수신 사용자와 선택 상품, 제안·약속 상태 이력
거래	transactions, transaction_history, transaction_checklists, user_ratings	상품별 거래와 상태 변경, 당사자 안전 점검·후기
운영	reports, report_history, notices, notifications	신고 처리·공지·알림과 관련 객체 링크
감사	admin_audit_logs, account_audit_logs, error_logs	관리자·개인정보 작업과 오류 추적

무결성 전략

- FK를 활성화하고 세션·이미지·이력 등 종속 데이터의 삭제 동작을 명시한다.
- 아이디는 UNIQUE와 대소문자 무시 인덱스를 함께 사용해 우회 중복을 차단한다.
- 상품 검색·메시지·알림·로그 테이블에 조희 패턴 기반 인덱스를 두고 페이지네이션으로 응답량을 제한한다.
- 거래 상태는 서버의 허용된 전이에서만 변경하며 모든 전이를 transaction_history에 append-only로 남긴다.
- 신고·채팅 제안·관리자 작업도 현재 상태뿐 아니라 history/audit 테이블을 함께 기록한다.

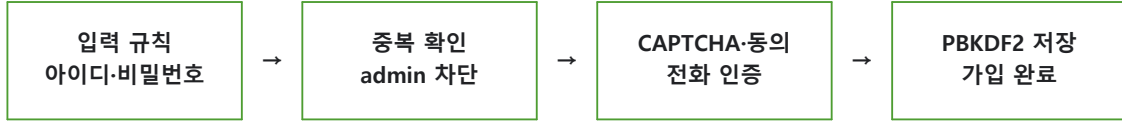
개인정보 생명주기

회원 탈퇴 전 비밀번호와 선택적 2단계 인증을 재확인한다. 계정은 참조 무결성을 위해 물리 삭제 대신 익명화하며, 상품은 비공개 처리하고 세션·찜·알림·검색기록을 제거한다. 로그인 로그의 IP·User-Agent도 비운 뒤 탈퇴 감사 기록을 보존한다.

04

인증과 계정 보안 흐름

회원가입



통제	설계
아이디	영문+숫자 5-20자, 대소문자 무시 중복, admin 예약어 차단
비밀번호	영문·숫자·특수문자 포함 8자 이상, 사용자별 16바이트 salt, PBKDF2-HMAC-SHA256
자동화 방지	서버 저장 일회용 계산 CAPTCHA, 5분 만료, 회원가입 Rate Limit
전화번호	국내 휴대전화 형식, 6자리 일회용 코드, 5분 만료·시도 횟수·재발급 제한
개인정보	수집 항목·목적·보유 기간 표시와 필수 동의 검증

로그인·잠금·복구

- 존재하는 계정의 비밀번호 실패만 누적하며 5회 실패 시 password_reset_required를 설정한다.
- 재설정은 아이디와 가입 전화번호가 모두 일치해야 코드를 발송한다. 알 수 없는 계정은 변경할 수 없다.
- 세션은 256비트 난수 토큰, HttpOnly, SameSite=Lax, 8시간 만료이며 HTTPS에서는 Secure를 강제한다.
- TOTP는 30초 시간창과 1칸 오차를 허용한다. 복구 코드는 원문 대신 SHA-256 해시를 저장하고 성공 즉시 사용 처리한다.
- 로그인 IP·User-Agent가 최근 성공 기록과 다르면 보안 알림을 만들고 사용자는 다른 모든 기기를 로그아웃할 수 있다.

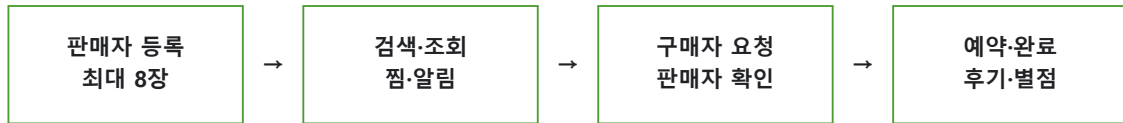
운영 관리자 계정

코드에 기본 비밀번호가 없다. 최초 실행은 WHITE_MARKET_ADMIN_PASSWORD가 정책을 만족해야 관리자 계정을 만들고, 첫 로그인 직후 새 비밀번호 변경을 강제한다. 운영 모드에서는 HTTPS·SMS Webhook·12자 이상 관리자 비밀번호·24자 이상 백업 키가 모두 없으면 서버가 시작되지 않는다.

05

상품·채팅·거래 흐름

상품 수명주기



상품 사진은 브라우저에서 축소-JPEG 재인코딩한 뒤 서버가 형식과 크기를 다시 검증한다. 대표 사진과 순서를 별도로 product_images 행으로 저장한다. 가격 인하 시 찜한 사용자에게 알림을 만들고, 알림 링크는 관련 상품 또는 채팅방으로 이동한다.

거래 상태 전이

현재	행위자	행동	다음 상태	서버 규칙
판매중	구매자	거래 요청	거래요청	자기 상품 금지, 중복 활성 요청 차단
거래요청	판매자	수락	예약중	상품도 예약중, 다른 요청 자동 거절
거래요청	판매자	거절	거래거절	상품은 계속 판매중
예약중	구매자	거래 완료	거래완료	상품도 완료, 후기 활성화

채팅 안의 제안과 증빙

- SSE로 새 메시지, 읽음 상태, 상대 접속 상태를 새로고침 없이 갱신한다.
- 가격 제안은 구매자가 금액을 만들고 판매자가 수락·거절한다.
- 거래 약속은 장소·시간을 제안하고 기존 약속을 superseded 처리한 뒤 변경안을 별도 기록한다.
- 거래 증빙 화면은 거래 당사자만 열 수 있으며 상태 이력·채팅·가격 제안·약속 변경을 한 번에 조회한다.
- 별점은 실제 채팅 관계 또는 완료 거래 당사자에게만 허용해 임의 평가를 차단한다.

06

위협 모델과 방어

위협	공격 영향	핵심 대응	근거
SQL Injection	DB 조회·변조	모든 값 파라미터 바인딩, 동적 컬럼 허용 목록	app.py:1905, 2476, 3762
XSS	세션 탈취·임의 요청	html.escape, textContent escape, strict CSP	app.py:165, 1577; chat.js
CSRF	사용자·관리 작업 위조	세션별 토큰, 중앙 POST 검증, SameSite	app.py:1249, 1445
계정 탈취	계정·거래 제어	PBKDF2, 잠금, 본인인증, TOTP·복구코드	app.py:453, 1891
권한 상승·IDOR	타 사용자·관리 데이터 접근	관리자/당사자/소유자 조건을 쿼리에 결합	app.py:1464, 4138, 4218
악성 이미지	저장소 고갈·브라우저 공격	Base64·크기·JPEG 구조 검증, 난수명, Rate Limit	app.py:213-315
백업 유출·변조	전체 개인정보 노출·훼손	AES-GCM, integrity_check, 재인증, 세션 폐기	app.py:335-375, 4658

보안 헤더와 전송 보호

CSP는 self 이외의 스크립트·스타일·이미지 연결을 제한하고 object-base-frame을 차단한다. X-Frame-Options: DENY, Referrer-Policy: no-referrer, X-Content-Type-Options: nosniff, COOP를 모든 응답에 적용한다. 운영 HTTPS에서는 HSTS와 Secure 세션 쿠키를 추가한다.

요청 횟수 제한

로그인·비밀번호 찾기·검색·채팅·신고·이미지 업로드·백업 복원은 기능별 한도와 시간차를 갖는다. IP 해시와 계정/사용자 ID를 함께 키로 사용해 IP 변경 또는 다계정 우회를 줄이고, 초과 시 Retry-After를 포함한 HTTP 429를 반환한다.

잔여 위험: 인메모리 Rate Limit은 단일 프로세스 범위이며 다중 인스턴스에서는 Redis 같은 중앙 저장소가 필요하다. 실 SMS 공급자 연결 시 API 키 비밀 저장소·공급자 서명 검증·전송 로그 마스킹을 추가해야 한다. 상세 위협 모델은 docs/threat-model.md에 기록했다.

07

자동 테스트와 보안 검사

실행 결과

검사	결과	확인 내용
unittest	13/13 성공	CSRF, 권한 상승, 채팅-거래 IDOR, SQLi, XSS, 잠금, 복구코드, Rate Limit, 헤더, 백업, 관리자, TOTP
Python compile	성공	app.py 문법 및 import 검증
JavaScript check	성공	chat.js 구문 검증
Bandit -lll	고위험 0건	TOTP SHA-1의 RFC 호환 예외만 nosec 주석으로 관리
Docker Compose	성공	운영 예시 환경으로 서비스-볼륨-Caddy 구성 파싱
민감정보 검사	통과	실제 관리자-백업 키, 예전 기본 비밀번호, 개인키 패턴 없음

핵심 보안 회귀 테스트

테스트	기대 결과
일반 사용자의 /admin 접근	403, 관리자 상태 변화 없음
다른 사용자의 거래 증빙 ID	404 또는 403, 이력 노출 없음
대화하지 않은 제3자 채팅 URL	제3자 메시지 내용 미노출
username에 SQL 조건 입력	인증 실패, 쿼리 구조 변경 없음
상품 검색에 script 태그 입력	HTML escape, 실행 가능한 태그 없음
비밀번호 5회 오류	계정만 잠금, 본인인증 재설정 요구
같은 복구 코드 두 번 사용	첫 사용만 성공, 두 번째 거부
암호화 백업 1바이트 변조	AES-GCM 인증 실패, DB 미교체

CI 구성

GitHub Actions는 push-pull_request-수동 실행에서 Python 3.12 환경을 만들고 의존성 설치, compile, unittest, Bandit 고위험 검사를 수행한다. PR에서는 dependency-review-action으로 의존성 변경 위험도 함께 확인한다.

08

화면 검증 - 데스크톱

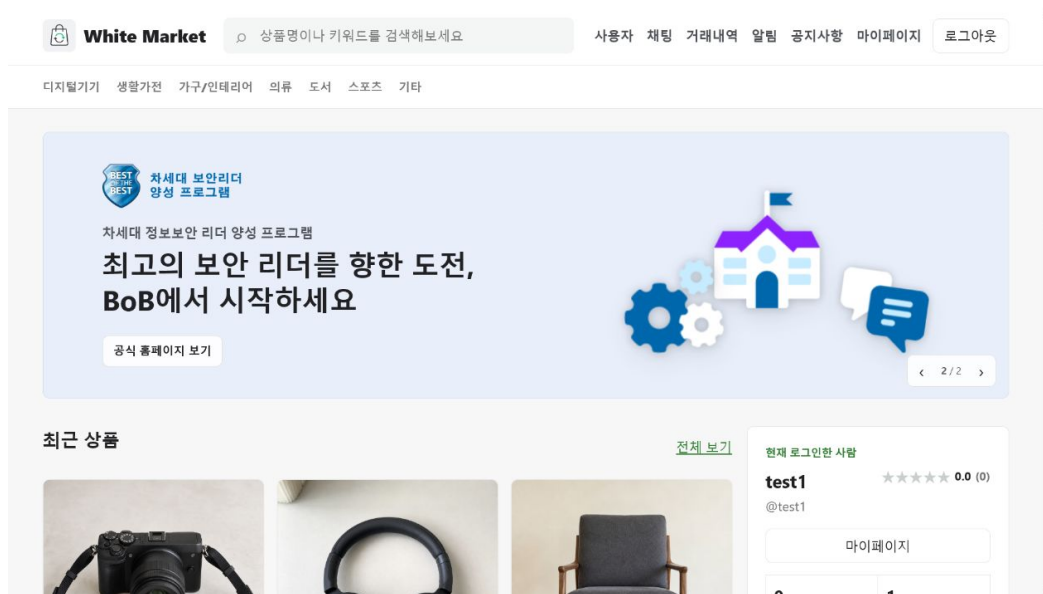


그림 1. 로그인 상태의 홈 화면. 광고 배너, 최근 상품, 계정 요약을 분리했으며 1280×720에서 가로 넘침이 없다.

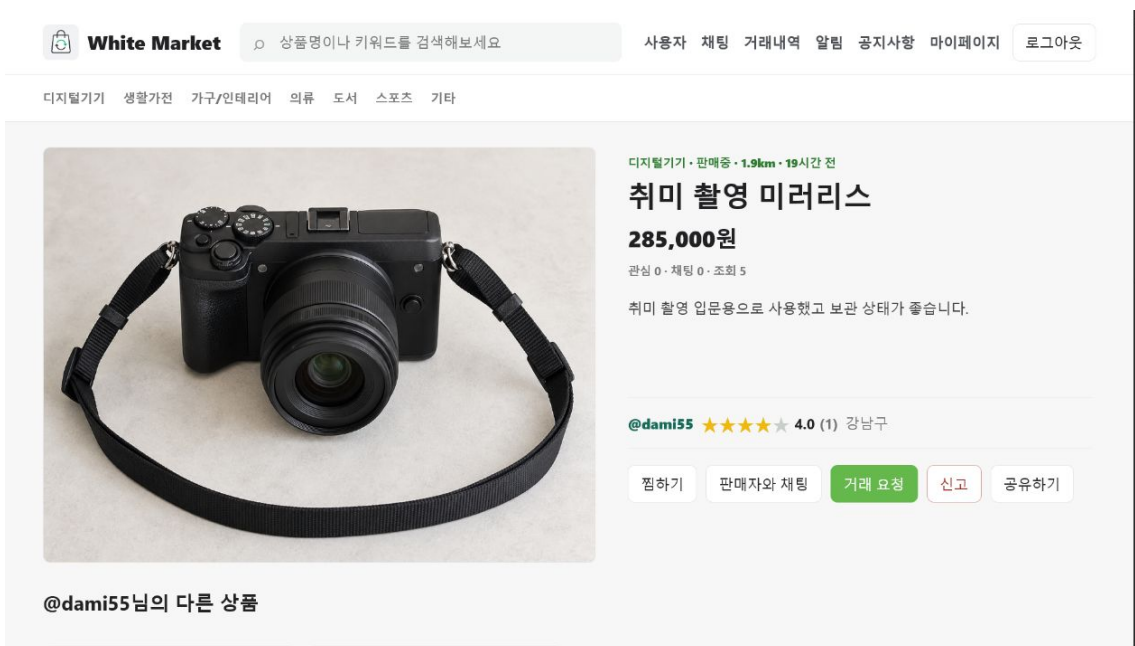


그림 2. 상품 상세. 사진 영역과 설명 영역의 경계는 32px 이상 떨어져 있으며 실제 좌표 검사에서 겹침이 없었다.

09

화면 검증 - 모바일

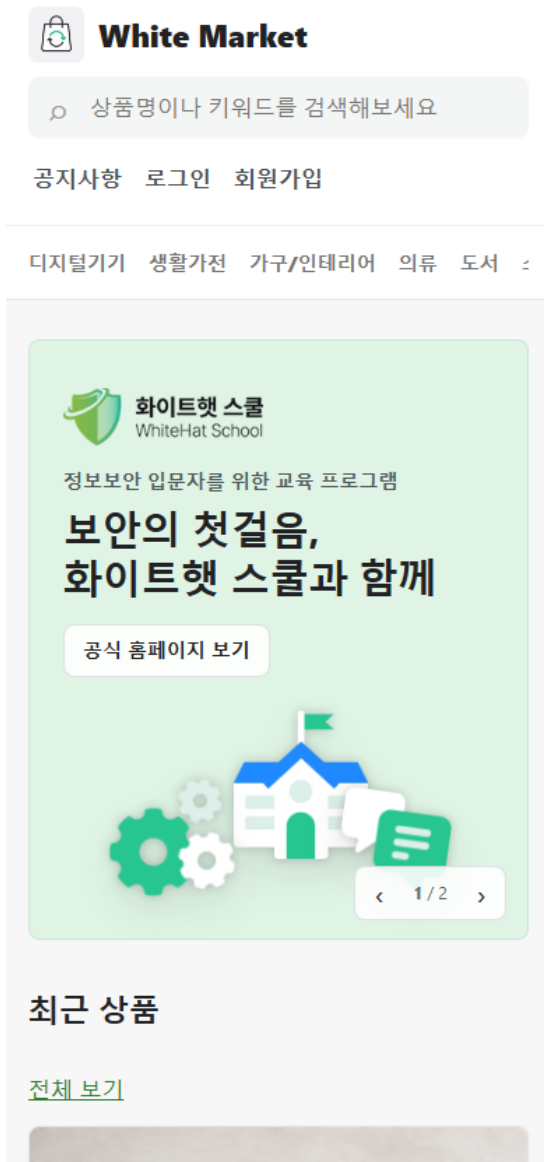


그림 3. 390px 홈: 1열 상품 카드, 가로 스크롤 없는 본문



그림 4. 390px 상세: 사진과 설명이 세로로 전환되어 겹침 없음

측정 결과: viewport 390×844, document overflow=false, 홈 grid 358px 1열, 상세 display=flex, gallery/info 각각 358px, 두 영역 overlap=false.

10

빈 상태·오류·채팅 사용성

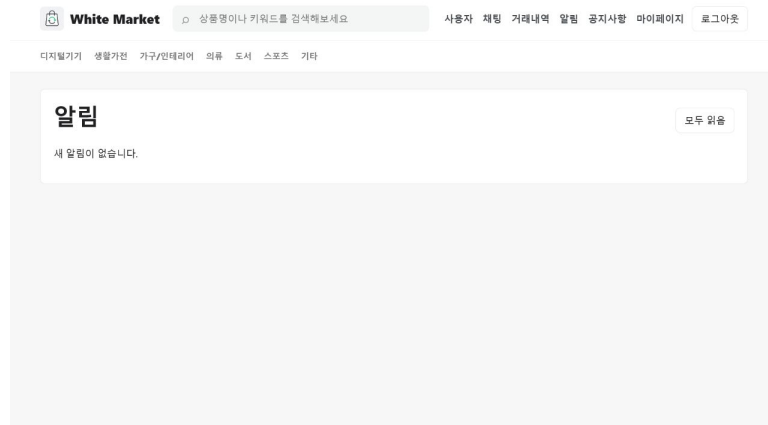


그림 5. 새 알림이 없을 때의 명시적 빈 상태. 사용 가능한 '모두 읽음' 행동은 유지한다.

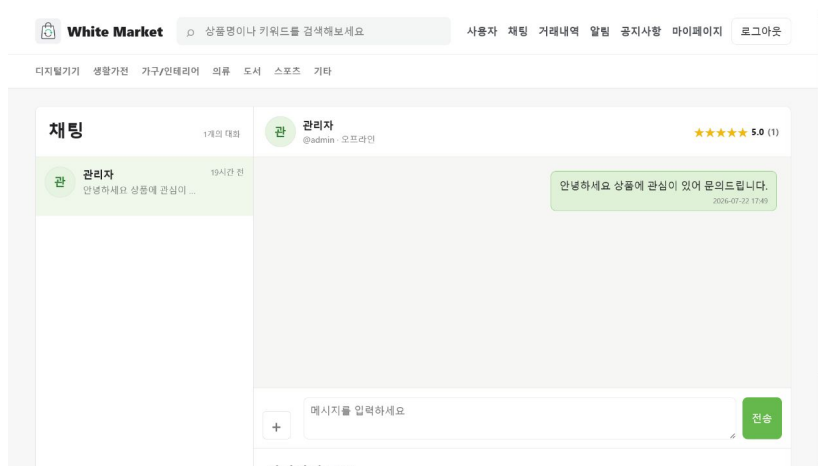


그림 6. 실시간 채팅. 목록·상대 프로필·메시지·입력 영역을 한 화면에 배치하고 긴 메시지는 anywhere 줄바꿈한다.

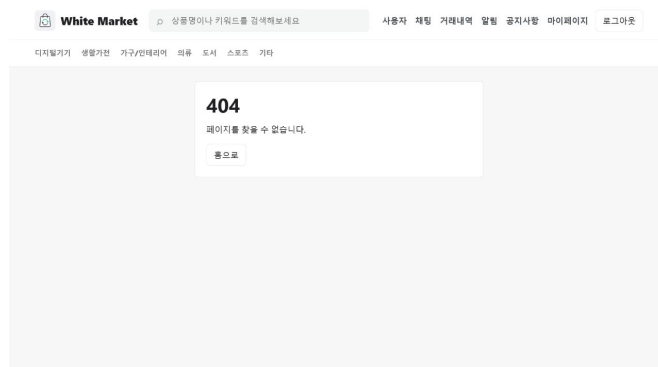


그림 7. 내부 정보 없이 상태 코드와 복귀 행동만 제공하는 404 오류 화면.

11

접근성과 최종 사용성 점검

점검	방법	결과
키보드	페이지 진입 후 첫 Tab	'본문 바로가기' 링크가 보이고 main으로 이동
포커스	:focus-visible 계산·시각 확인	3px 녹색 외곽선과 3px offset
폼 이름	로그인 input의 label/aria/placeholder 검사	검색·아이디·비밀번호·2FA 모두 접근 가능한 이름 보유
이미지	상품·채팅·프로필 이미지 alt 검사	의미 이미지 설명 제공, 버튼 내부 썸네일은 버튼 aria-label로 설명
긴 상품명	공백 없는 문자열 반복 삽입	페이지·제목·카드 가로 overflow 모두 false
모션	prefers-reduced-motion	카드·하트 transition과 smooth scroll 비활성화
로딩	채팅 비동기 전송	버튼 비활성화, '전송 중', form aria-busy 후 원상 복구

상태별 UX

- 빈 상품·찜·최근 본 상품·알림·신고 목록은 빈 영역 대신 현재 상태를 문장으로 보여준다.
- 서버 오류는 요청 ID로 운영 로그와 연결하되 사용자 화면에는 traceback, SQL, 환경 변수를 노출하지 않는다.
- 이미지 처리 중에는 선택 수와 처리 상태를 표시하고, 채팅 전송 중에는 중복 제출을 막는다.
- 좁은 화면에서 헤더 탐색은 가로 스크롤 가능한 한 줄로 유지하고 상세 행동 버튼은 안정적인 그리드로 바뀐다.
- 상품 상세 사진은 aspect-ratio와 minmax 그리드를 사용해 이미지 크기 변화가 설명 영역을 밀어내지 않게 했다.

QA는 실제 서버와 Chromium에서 수행했다. 데스크톱 1280×720, 모바일 390×844 모두 document overflow=false 였고, 상품 상세의 gallery와 product-info 교차 영역도 false였다.

12

운영 배포 연습

구성

파일	역할
Dockerfile	Python 3.12, 비루트 사용자, 정적 소스와 데이터 경로 분리
compose.yaml	앱·Caddy, 영속 DB/백업/업로드 볼륨, 80-443 노출
deploy/Caddyfile	도메인 기반 자동 TLS 인증서와 app:8010 reverse_proxy
.env.production.example	도메인·관리자 비밀번호·백업 키·SMS·지오코딩 설정 목록
deploy/README.md	DNS·방화벽·기동·헤더·로그 점검 순서

운영 시작 전 강제 조건

WHITE_MARKET_ENV=production일 때 HTTPS 플래그, HTTPS SMS_WEBHOOK_URL, 12자 이상 관리자 초기 비밀번호, 24자 이상 백업 키를 검사한다. 하나라도 없으면 시작 단계에서 RuntimeError를 발생시킨다. 따라서 로컬 개발 모드처럼 인증번호를 화면에 표시하는 구성은 운영 서버에서 실행할 수 없다.

실제 배포 순서

1. 도메인 DNS를 서버로 연결
2. .env.production을 비밀 저장소에서 주입
3. SMS 공급자의 HTTPS API/Webhook 연결
4. docker compose config 및 up -d --build
5. HTTPS-HSTS-CSP-Secure 쿠키 확인
6. 관리자 최초 비밀번호 변경과 암호화 백업·복원 훈련

실습 한계: 실제 도메인과 SMS 공급자 계약 정보가 없으므로 외부 인증서 발급과 SMS 전송은 실행하지 않았다. 대신 운영 설정 누락 차단, Docker Compose 파싱, Caddy 구성, 데이터 영속화까지 로컬에서 검증했다.

13

Git·GitHub와 제출 구조

저장소 정책

포함	제외
app.py, static/, tests/, docs/, deploy/, Dockerfile, compose.yaml, Actions	market.db, backups/*.*.db*.wmbak, *.log, __pycache__
.env.production.example처럼 값이 없는 템플릿	.env, .env.production, local-security-env.ps1
기본 상품·광고·브랜드 정적 자산	사용자 프로필·상품·채팅 업로드 이미지

업로드 전 점검

- 알려진 실제 관리자·백업 키와 개인키 헤더 패턴을 전체 추적 후보에서 검색해 일치 없음 확인
- 예전 기본 관리자 비밀번호 문자열과 호환 마이그레이션 코드를 제거
- git check-ignore로 DB·백업·로컬 환경 파일·로그가 제외되는지 확인
- 커밋 직전 staged 파일 목록을 검토하고 GitHub에는 비공개 저장소로 생성
- push 후 Actions의 compile-13개 테스트·Bandit 결과를 확인

저장소 링크

<https://github.com/i-Zer0/white-market-secure-coding>

재현 명령

```
python -m pip install -r requirements.txt
$env:WHITE_MARKET_ADMIN_PASSWORD='강력한 초기 비밀번호'
$env:WHITE_MARKET_BACKUP_KEY='별도의 24자 이상 백업 키'
python app.py
python -m unittest discover -s tests -v
```

14

결론

White Market은 강의 자료의 필수 기능을 출발점으로 회원·상품·소통·차단·검색·관리 기능을 구현하고, 거래 상태·후기·알림·보안 관리·운영 안정성까지 확장했다. 기능이 많아질수록 서버가 권한과 상태를 결정하도록 설계해 클라이언트 조작이 데이터 무결성으로 이어지지 않게 했다.

달성 결과

- SQL Injection, XSS, CSRF, 계정 탈취, 무차별 로그인, 권한 상승·IDOR, 악성 이미지, 백업 유출·변조에 대한 방어와 테스트 근거를 확보했다.
- 거래·신고·관리·복구·백업의 중요한 변경은 현재 값뿐 아니라 별도 이력과 감사 로그로 보존한다.
- 운영용 비밀과 개발용 편의 기능을 환경 모드로 분리하고, HTTPS·SMS·백업 키 누락 시 운영 시작을 차단한다.
- 모바일·긴 텍스트·빈 목록·오류·키보드 접근성까지 실제 브라우저에서 검증했다.
- GitHub Actions로 테스트와 정적 보안 검사를 재실행할 수 있는 제출 구조를 만들었다.

향후 개선

실서비스 단계에서는 Redis 기반 분산 Rate Limit, 중앙 비밀 관리자, 공급자 서명형 SMS 연동, WAF·중앙 로그·모니터링, 정기 복구 훈련, 파일 악성코드 스캔, 다중 프로세스 애플리케이션 서버를 추가한다.

기능을 많이 만드는 것에서 끝나지 않고, 누가 어떤 상태에서 무엇을 할 수 있는지 서버가 증명하도록 만든 프로젝트다.

부록 파일

docs/threat-model.md · tests/test_app.py · .github/workflows/security.yml · deploy/README.md